Evidence-Governed Agent Harnesses

Making Evidence Survive Long-Running Agent Execution

What should an agent remember after a crash at step 40 of 60?

Not just its state — but what it verified, what it accomplished,

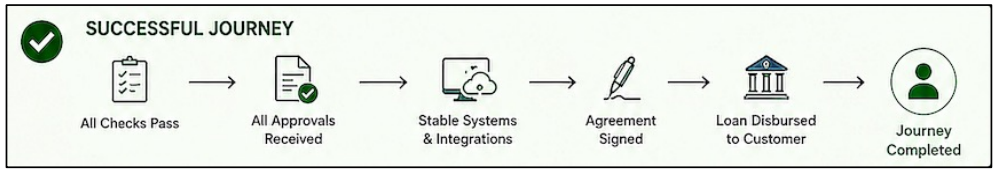
and what it is still allowed to do.

Khaja Shaik

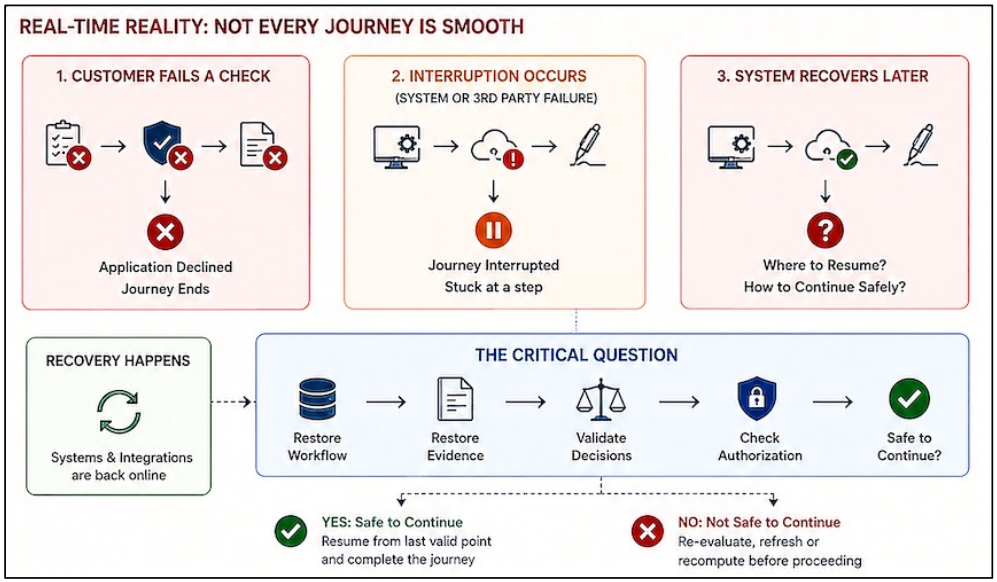
Fifth Elephant Hyderabad | 2026

End-to-End Loan Journey

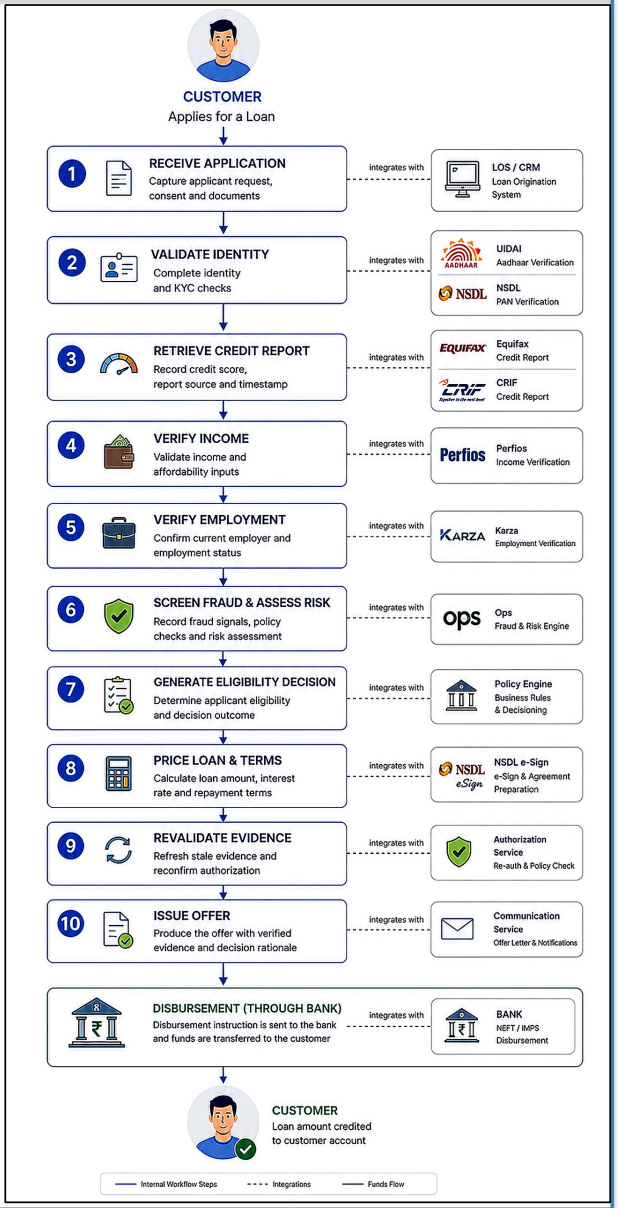
Customer must complete entire process successfully to receive disbursed amount.



Can system safely continue with evidence information collected before crash/ restart ?..

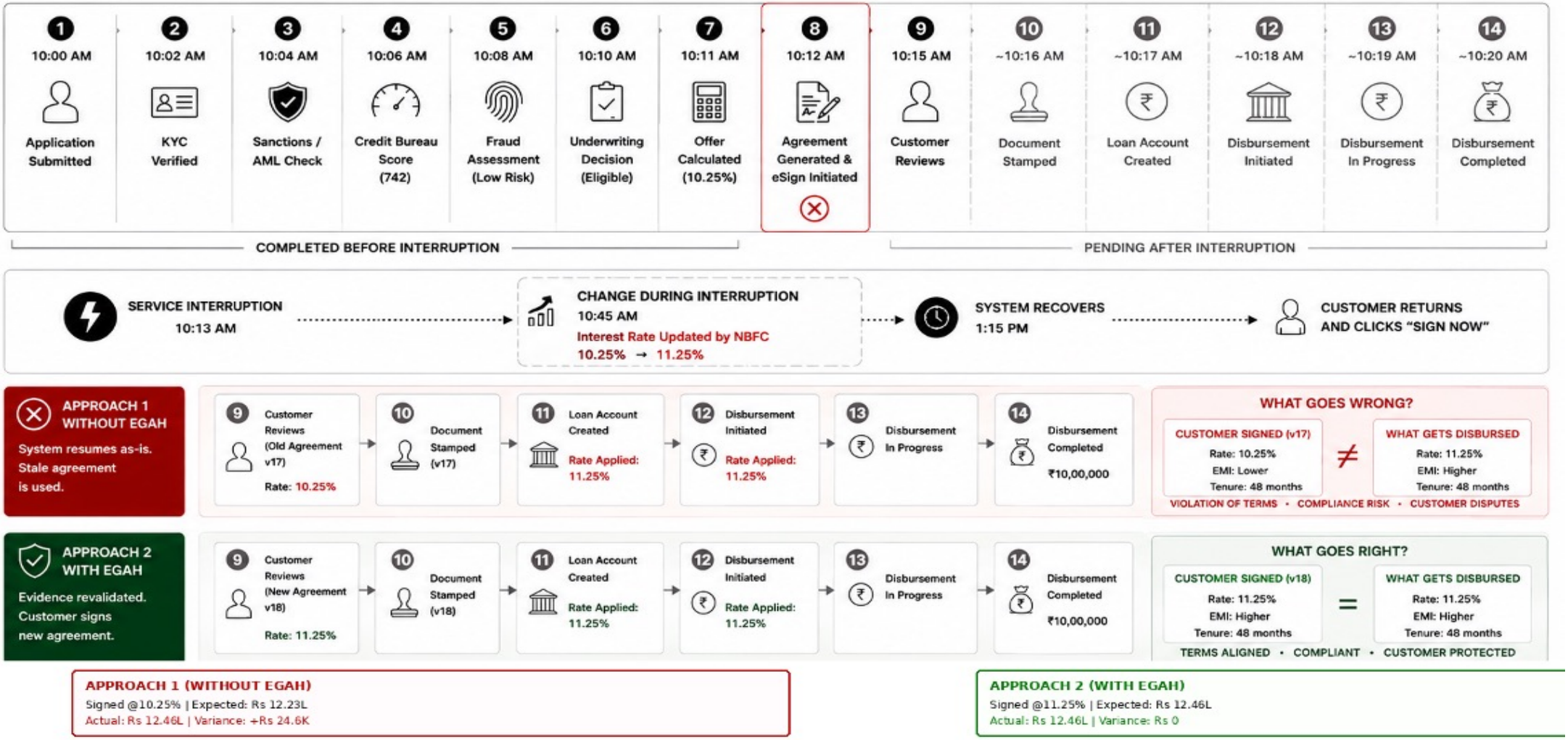


Execution State ≠ Evidence State : Successful State Recovery Does Not Imply Safe Agent Continuation



Loan Journey Interruption at e-Sign stage

Interest rate changed during interruption from 10.25% to 11.25%



When Resuming Execution Is Not Enough

WHAT is the problem? → WHY does it matter? → HOW does EGAH address it? → WHERE does it apply? → WHO needs it?. Harness provides reusable, cross-cutting governance

What traditional applications address workflow by workflow. EAGH governs consistently through a shared harness layer.

EGAH is run-time governance layer that persists/ensures that an AI agent resumes with valid evidence not just recovered state.

WHAT	WHY	HOW	WHERE/ WHO
State can be recovered but decision basis may not still be valid.	Evidence may expire or change while execution is interrupted	Restore and validate decision basis before continuation.	AI agents & long-running workflows where actions matter.

a. Durable Execution State: Where are we?

b. Durable Evidence State: What justifies us being here?

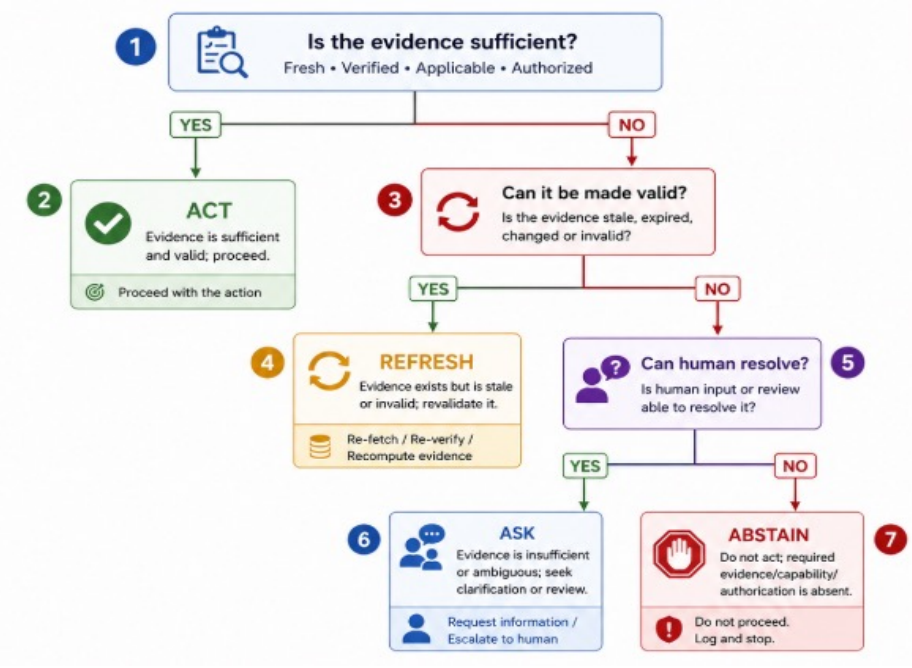
c. Policy / Decision Control: What are we allowed to do next?
ACT → REFRESH → ASK → ABSTAIN

ACT: Evidence is sufficient and valid; proceed

REFRESH: Evidence exists but is stale or changed; revalidate it

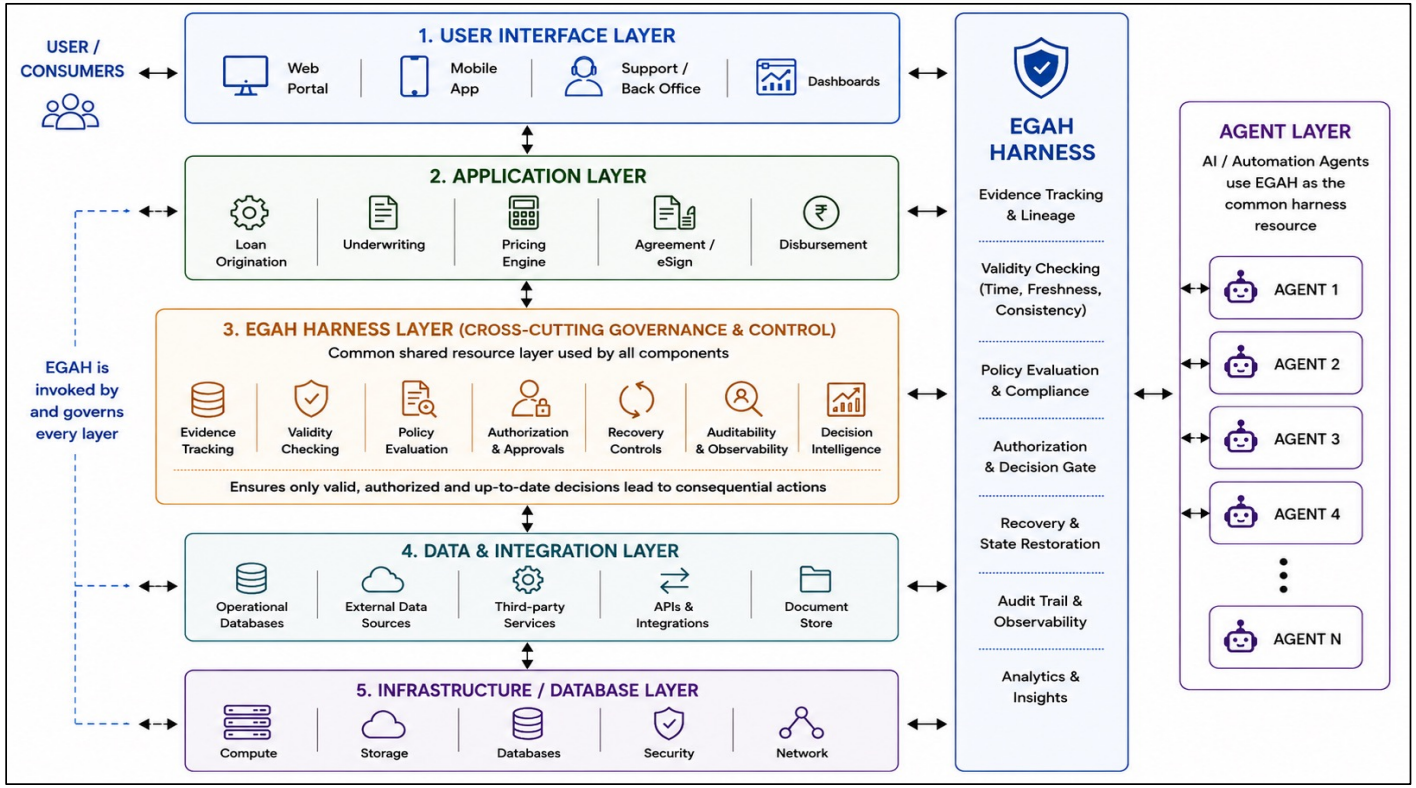
ASK: Evidence is insufficient or ambiguous; seek clarification or human review

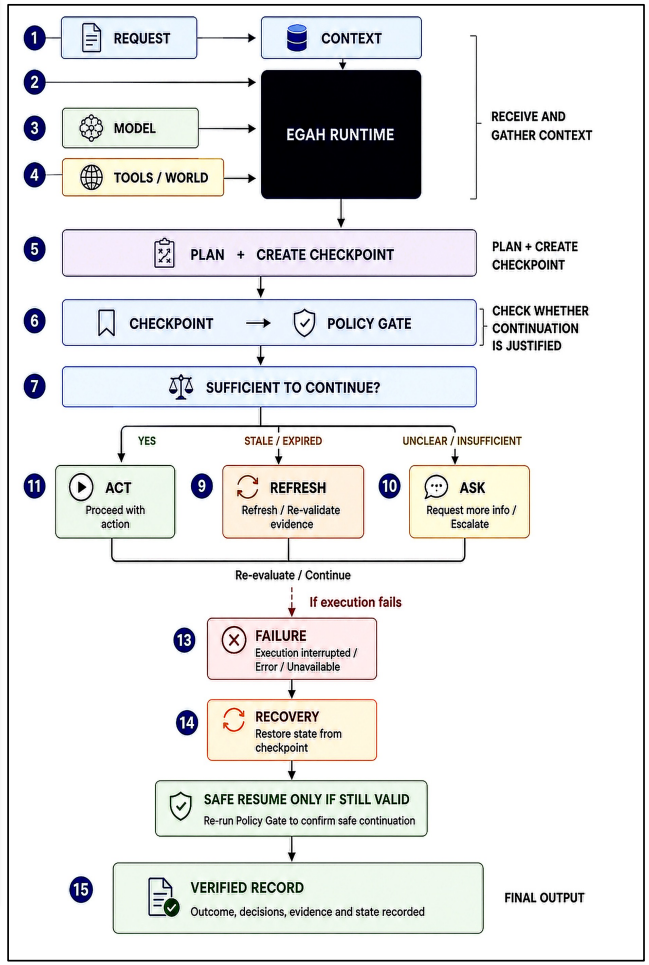
ABSTAIN: Do not act; required evidence/capability/authorization is absent



Architecture

End-to-end evidence-governed control layer between intelligent decision-making and consequential action





EMBEDDED SDK

Python library around consequential nodes; store + decide.

RUNTIME HOOKS

LangGraph, Temporal, Dapr or Prefect; SQL/NoSQL evidence stores.

SERVICE + EVENTS

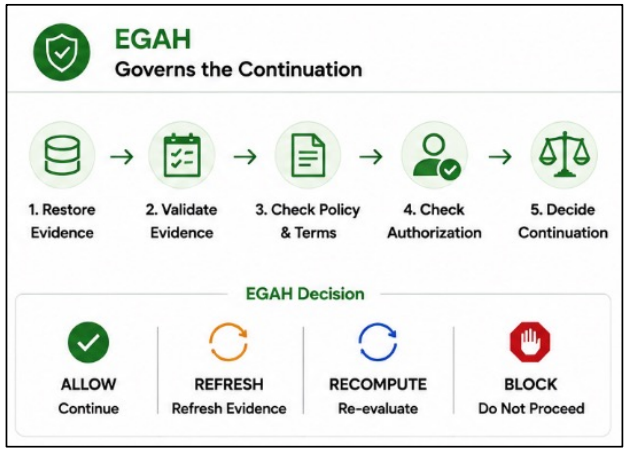
REST/gRPC; OTel export; OPA/Cedar; bureau, KYC,fraud and pricing adapters.

Observability vs EGAH

Knowing What Happened to Governing What's Next. Telemetry gives EGAH inputs which leverages verified information into runtime governance state.

EAGH sits on top of observability, leveraging traces/telemetry with evidence to validate outcome/ continuation.

Type	Observability	EGAH
Primary question	What happened ?.	Can we safely continue ?.
Focus	Execution visibility	Evidence-governed action
Tracks	Logs, traces, metrics, events	Evidence, validity, provenance
After a crash	Where did workflow stop?.	Is previous decision still valid?.
Detects	Failures and anomalies	Valid, Stale/Invalid or insufficient
Recovery role	Helps diagnose and restore	Controls continuation allowed?
Output	Insight/ Visibility	ALLOW/ REFRESH/ RECOMPUTE/ BLOCK
Output	System failed/recovered during e-sign stage	Validate before allowing continuation



EGAH complements OSS stack

EAGH makes the **decision to continue itself evidence-governed and policy-controlled**

Existing frameworks answer

Can the workflow resume?

EAGH additionally answers

Should the workflow resume, based on evidence that is still valid, fresh, authorized and sufficient?

Capability	Existing OSS(Open Source Software) strength	Why it is Not Sufficient Alone
LangGraph	Graph checkpoints, replay, HIL	Restores workflow state, but does not explicitly govern whether prior evidence is still valid
Temporal	Durable history and replay	Knows execution history, but not whether external facts have become stale
Dapr Workflow	Retries; signed history	Handles execution recovery, but not the lifecycle of evidence behind a decision
Prefect	Flow/task state and retries	Can resume execution, but continuation may require renewed authorization
OpenTelemetry	Traces, metrics, logs, baggage	Observes what happened, but does not decide whether it is safe to continue
Langfuse / Phoenix	LLM traces, evals, cost, latency	Provides observability, but does not maintain durable dependencies between evidence and consequential decisions
EGAH	Composes above any runtime	ACT · REFRESH · ASK · ABSTAIN

Impact: Operations & Risk Outcomes

State-only vs EAGH enabled operating model

Unsafe Resumes Allowed
156 (78%) State-only → 0 (0%) EAGH

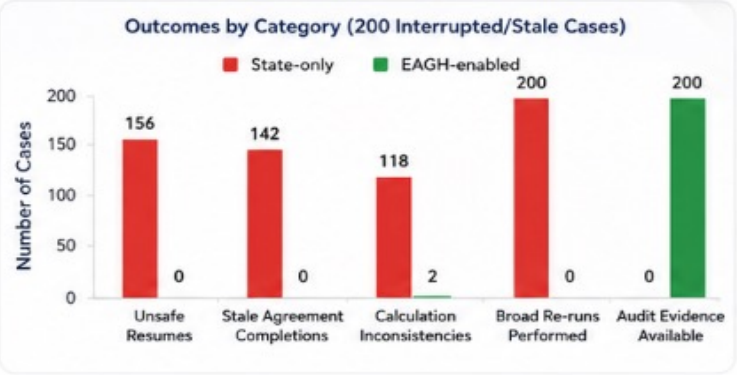
Stale Agreement Completions
142 (71%) State-only → 0 (0%) EAGH

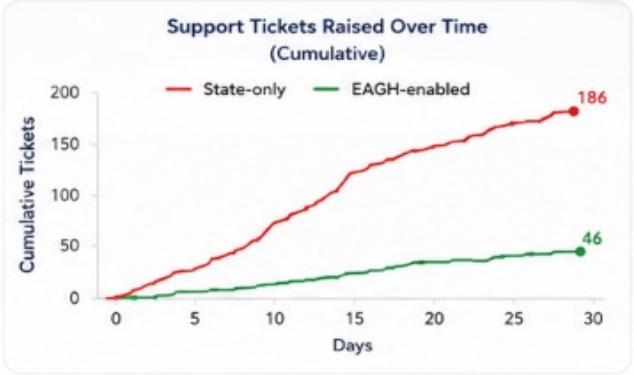
Loan Calculation Inconsistencies
118 (59%) State-only → 2 (1%) EAGH

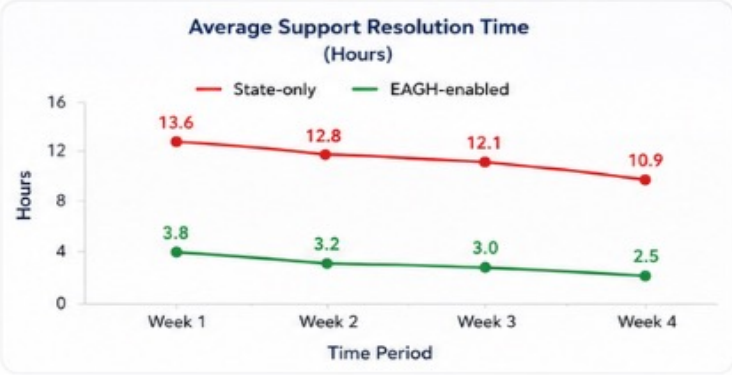
Support Tickets Raised
186 (93%) State-only → 46 (23%) EAGH

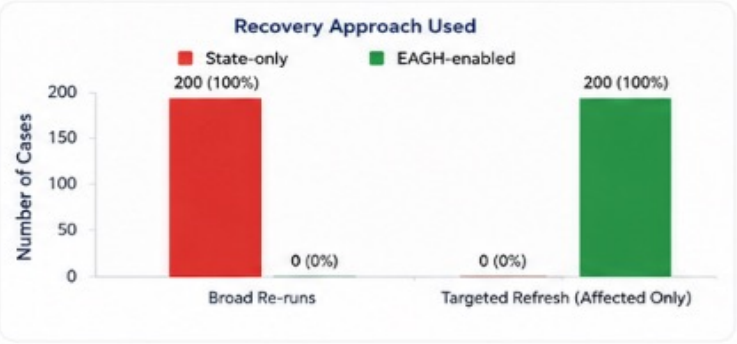
Avg. Support Resolution Time (per ticket)
12.4 hrs State-only → 3.1 hrs EAGH

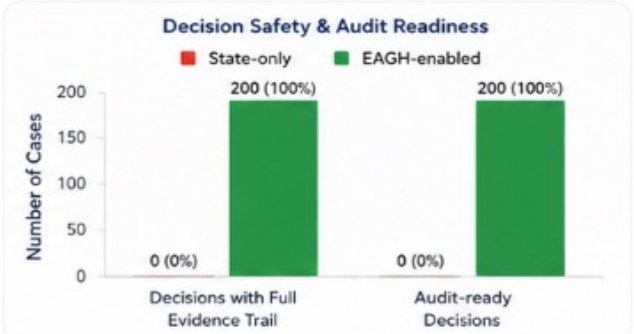
Manual Recovery Interventions
168 (84%) State-only → 28 (14%) EAGH

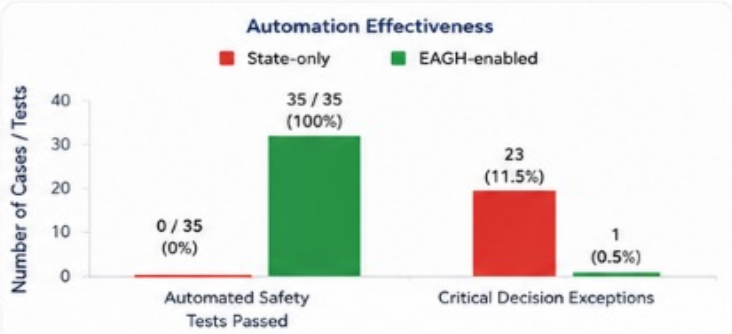












Learnings & Limitations

Key insights gained, current limitations, and areas for future evolution

Learning	What the PoC exposed	Next engineering step
Evidence Freshness	TTL alone cannot determine validity	Support events, versions and authorization checks
Selective Checkpointing	Checkpointing every step is wasteful	Place at consequential decision boundaries
Dependency Awareness	Stale inputs invalidate downstream decisions	Maintain evidence dependency relationships

Limitations	What the PoC exposed	Next engineering step
Live Staleness Detection	Aging simulated by workflow position	No real-time invalidation from providers
Performance	~28% median latency overhead	Checkpointing must remain selective
Scope Fit	Best for deterministic/predictable risky work-flows	Low-risk workflows may not justify EAGH overhead